

Machine Learning

Mikkel Moth Billing
202307989

28. august 2025

Indhold

Indhold	1
1 Linear model	2
1.1 Decision Trees	3
Entropy	4
Loss Function	5

1 Linear model

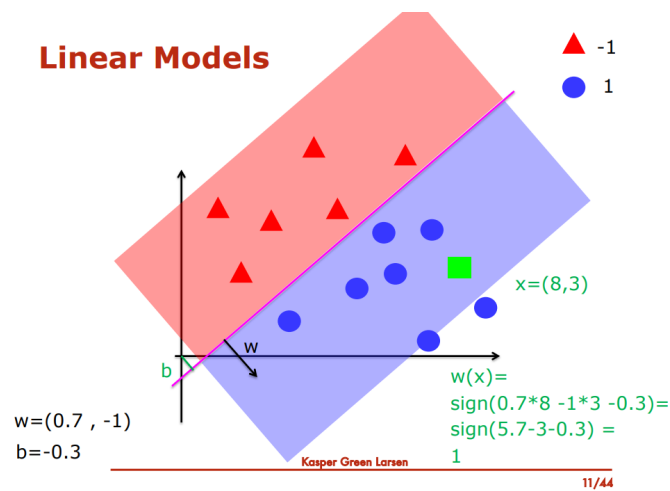
Hypothesis consists of the features vector:

$$x = \{x_1, \dots, x_d\}$$

and a bias b . If the linear combination of the weights and the input, and compare it to the bias, we can make predictions.

Example of mortgage, approve is 1 and decline is -1 . So we can predict that we approve

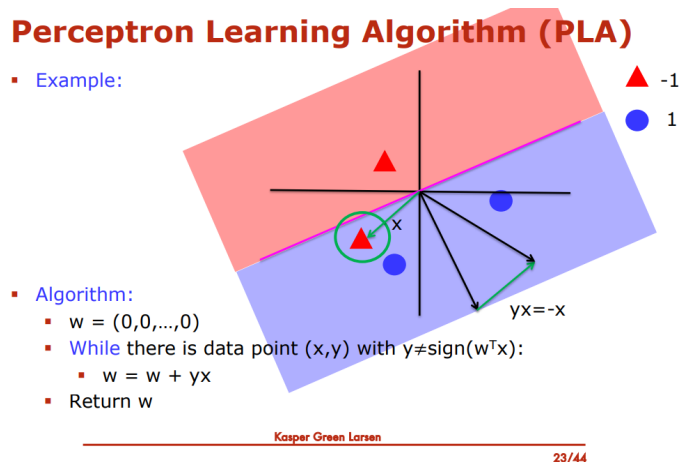
$$w_1x_1 + \dots + w_dx_d > -b$$



To find the parameters we can use **Perceptron Learning Algorithm (PLA)**.

Algorithm: While there is a data point (x, y) with

$$y \neq \text{sign}(w^T x) \rightarrow w = w + yx.$$



As we can see the algorithm can enter an infinite loop, as there might not be a linearly separable dataset.

A way to fix this is to use Pocket PLA which saves the most accurate hyperplane.

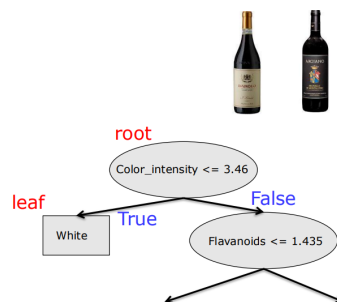
Pocket PLA

- Would like to still do something useful if not linearly separable!
- Pocket version: Remember best w in your pocket.
- Algorithm:
 - $w = (0,0,...,0)$
 - $bestScore = 0$
 - $bestW = w$
 - While you have more time, let (x,y) have $y \neq \text{sign}(w^T x)$:
 - $w = w + yx$
 - $score = \text{percentage of data correctly classified by } w$.
 - If $score > bestScore$:
 - $bestW = w$, $bestScore = score$
 - Return $bestW$

1.1 Decision Trees

Decision Trees

- Classifying wine.
 - Red, White, Rosé
- Predict(x):
 - Start at root node.
 - Test condition in root and descent to relevant child.
 - Upon reach leaf:
 - Return value stored at leaf.
- Node:
 - Question tests **one** feature against **one** value.

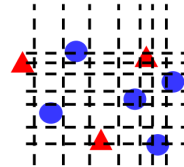
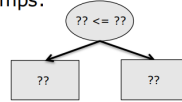


We would like to keep the decision tree small to not overfit to the training data. So we could find the best performing tree with k nodes. But this is **NP-hard**, so we rely on **heuristics**. With a greedy learning algorithm with $k = 2$:

Small Decision Trees

- Greedy Learning Algorithm.

- Decision Stumps:



- Find best decision stump:

- For each of d features:
 - For each of $n-1$ relevant splits:
 - Try that root, partition points into leaves, assign best leaves, compute accuracy.
- Use best split found.

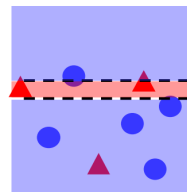
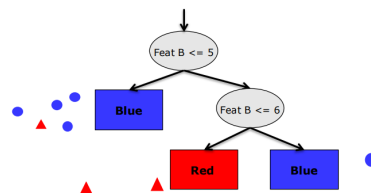
We test the tree on all possible split, this has $O(n^2d)$ runtime.

Another way to build a tree is to increase k to maximise the accuracy.

Small Decision Trees

- Final Tree:

$k=2$ example



Accuracy: 7/8

Entropy

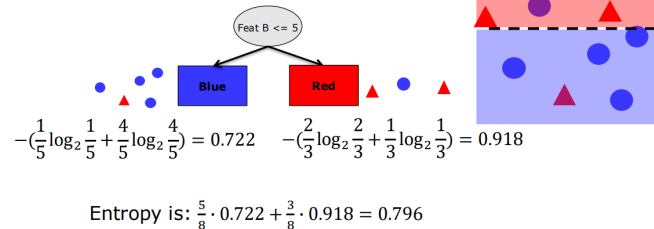
Another way to test decision tree is to base it on entropy. If we have n point with n_i as the numbers of points with label i . And $p_i = n/n_i$. Then entropy is:

$$H(S) = - \sum_{i=1}^k p_i \log_2 p_i.$$

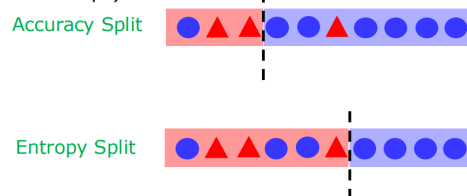
Entropy is a measure of uncertainty in distributions, high entropy is high uncertainty. $H(S)$ describes the expected number of bits it takes to describe the label of a random element drawn from S . If all elements in S are equal $H(S) = 0$. If there is an equal amount $H(S) = \log_2 k$.

Entropy

- Entropy of tree: $\sum_v \frac{|S_v|}{n} H(S_v)$



- May give different trees than using accuracy.
- 1-dimensional example, i.e. one feature.
 - Entropy prefers "pure" splits over accuracy.
 - Seem to work best in practice when constructing decision trees (larger than stumps).



Loss Function

A Loss Function weighs how much a mistake influences a model. We can then try to minimize the loss function over the training points. An example is the Least Square Loss Function:

$$L(y', y) = (y' - y)^2.$$

Small Regression Trees

- Greedy Learning Algorithm.
 - Start in simple case:
 - Our tree may only contain 1 node.
 - Root is a leaf.



- What is best choice of root?
 - The x that minimizes $\frac{1}{n} \sum_{i=1}^n (x - y_i)^2$ is the mean:
 - $x = \frac{1}{n} \sum_{i=1}^n y_i$.

Mean is $(2+3+9+10+11)/5=35/5=7$

Loss is $((2-7)^2 + (3-7)^2 + (9-7)^2 + (10-7)^2 + (11-7)^2)/5=14$.

